DPH Engineering Guide

Delphi Project Handoff

Windows desktop experimental software

Last changed 11 October 2026

For a Delphi developer building maintaining and validating DPH

This guide describes the implemented application and its evidence limits. Use the phase acceptance records for exact measured results and unresolved release gates.

Contents

Architecture and boundaries....3
Rework and acceptance status....3
Build and deployment....3
Input acquisition and project scope....4
Syntax contracts and limits....4
Symbols and typed relationships....4
Control flow and impact....5
SQLite generations and durability....5
Evidence and immutable history....5
Background coordination and cached views....6
FMX navigation and visualization....6
Curated knowledge and review....6
Canonical publication and privacy....7
HTML guides and bundle validation....7
Translation boundary....8
Verification and release discipline....8
Current semantic cache and extended evidence....8
Current schema and authored document contracts....9
Native review and learning projections....9
Saved observation persistence....9
Regression commands and evidence....10
Rebuild and inspect these manuals....10
Business glossary implementation and checks....11
Offline translation and localized publication....11
Native validation and release evidence....12
Structured project profile contract....12
Quality analysis and revision comparison....13
Source and revision evidence....13

Architecture and boundaries

DPH is an independent Delphi FMX application. It has no converter runtime or build dependency. The pinned VCL2FMXConverter v6.0.0 corpus is test input only, at commit e1fd10b2d3b06f2fc3f726b8eb7f234a54faf63d. No converter source was copied into the current analyzer. Preserve the original planning documents and global agents.md byte for byte.

The pipeline is selected project, bounded inventory, read-only byte acquisition, lexical and conditional processing, Pascal and form IR, semantic relationships and control flow, revision-consistent SQLite capture, cached workspace projection, curated knowledge and derived publication. Contract data remains separate from FMX. Every unsupported construct or unavailable dependency must remain diagnosable rather than being promoted into false completeness.

Application controls are serialized in DPH.UI.Main.fmx, DPH.UI.ProjectProfile.fmx and DPH.UI.Glossary.fmx and editable in the IDE. List contents and paint-box geometry are data projections. No scanned component class is instantiated. Avoid adding runtime-only controls, Pascal class helpers or an alternative authoritative store without explicit authorization.

Rework and acceptance status

The owner rejected the earlier WP7 delivery as inadequate for learning and handoff. The statement that implementation through WP7 was complete has been withdrawn. docs/Rework_Requirements_Audit.txt compares the original plan with the implemented product and outstanding work. Historical WP reports remain historical evidence, not acceptance of the reworked experience.

The rework adds a first-launch Getting started course, deterministic explanation engine, guided learning workspace, source-comment context, event traces, labeled and collapsed procedure-flow views, attributed screenshot imports and richer project-specific publication. Run tools/build-learning.cmd with an optional pinned reference database path for dedicated learning and real-WebView2 tests. The source-based first-time-programmer exercise still needs a human participant.

The implementation tracker records expanded semantic reuse, resource interpretation and retention/continuity controls. Their documented bounds still apply. Source safety, privacy, attribution and exact generation links are retained. Do not turn an automated pass count into a claim that every requirement or the human onboarding goal is finished.

Build and deployment

The installed compiler is RAD Studio 37.0.59082.6021. Load C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvars.bat before compiling. tools/build-wp0.cmd builds DelphiProjectHandoff.dproj with elevated MSBuild and runs the FMX resource smoke test. Output is bin/Win32/Debug, with DCUs in dcu/Win32/Debug. The initial executable target is Windows Win32.

Deploy the executable, all nine immutable migration files, and the architecture-matched WebView2Loader.dll. The reviewed Microsoft WebView2 SDK package is pinned to 1.0.4258.31; retain tools/WebView2/LICENSE.txt with binary redistribution. The browser loads the DLL by its absolute executable-folder path and activates Edge only when preview is requested. There is no intended IE fallback.

The repository standard folders include bin, dcu, docs/guides, docs/pdf, export, help, images and icons, samples, source distributions and tools. Deploy help/index.html for first-launch education. Reference source

and generated test databases remain ignored. Use the explicitly approved Python 3.13 executable for project tooling and the installed LibreOffice, never Microsoft Word.

Input acquisition and project scope

DPH.Project.Inventory follows the selected project, explicit references, uses clauses, include paths and approved external roots. Project XML is passive: targets and imported build scripts are not executed. Canonical paths are project relative and normalized; traversal, ambiguous macros, reparse paths, cycles and oversized acquisition are rejected or diagnosed. A repository-wide count of Pascal files is not the build scope.

DPH.Project.Source opens input read only, checks the final opened Windows handle path, captures bytes, records SHA256 and verifies decoding by round trip. Unicode byte offsets, line endings and explicit legacy encodings remain in the evidence contract. The maximum acquired source is 16 MiB. Binary forms are parsed as data; unsupported resources remain gaps.

A source viewer must re-acquire the live file and match the captured hash before using a cached range. Externally referenced sources use their explicit canonical inventory identity. Never invent a neighboring source path from a cached filename or execute input to fill an analysis gap.

Syntax contracts and limits

DPH.Parse.Model defines IR schema 1, analyzer DPH-WP2-5, token and node limits, source ranges and diagnostics. Lexing preserves original tokens and conditional information. Preprocessing evaluates supported expressions, explicit defines and includes while retaining uncertainty for unknown branches. An unavailable compiler symbol is not automatically a false condition.

The Pascal parser extracts declarations, routines, anonymous methods and supported statements into IR. Declaration and implementation occurrences remain distinguishable before semantic unification. The form parser accepts supported text and binary DFM/FMX structures without loading component packages. The inventory reads relevant project configuration and resources through bounded adapters.

Current bounds include 250000 tokens, 100000 IR nodes, nesting 128 and 10000 diagnostics. Opaque assembler, malformed syntax and unsupported grammar are partial. Tests use exact golden IR, encoding edge cases, cyclic includes, passive malicious imports and changed source. Maintain source byte ranges and diagnostics when extending support.

Symbols and typed relationships

DPH.Analysis.Model defines semantic schema 1 and analyzer DPH-WP3-11. Stable symbol keys contain qualified identity, signature and conditional variant. DPH.Analysis.Symbols unifies supported routine prototypes and bodies, resolves cross-unit declarations, typed receivers and suitable overloads, associates form events and records anonymous callbacks.

Relationship certainty is confirmed_static, possible_dynamic, inferred, runtime_observed or unresolved. Passive scanning never claims runtime_observed. A uses dependency is not a call. Known lexical calls, serialized bindings and declared types carry derivation rules and source ranges. Recognized asynchronous API spelling can establish a possible scheduling relationship without proving execution order.

Method pointers, unresolved receivers, dynamic dispatch, generated types and framework behavior must retain uncertainty. Basic reads, writes, callback assignment, static SQL spelling and report-model rendering

are conservative analyses. Renames are distinct identities until continuity is reviewed; offsets alone must not reassign knowledge.

Control flow and impact

DPH.Analysis.Flow builds per-routine control-flow nodes and typed edges for supported sequences, branches, loops, returns and exception/finally paths. Routine identity, source hash, range, node kind and expression remain available to inspectors and exports. Flow is bounded to 100000 nodes, 300000 edges and nesting 128.

Impact traversal is possible reachability through supported relationships. Do not label it proved runtime impact. The UI bounds a displayed graph to 120 nodes and 960 edges, while the stored semantic graph is the fuller evidence model. Uncertain edges remain visually distinct. Unsupported flow does not become a fabricated successful path.

SQLite generations and durability

TDPHKnowledgeStore owns one FireDAC SQLite connection on its creating thread and serializes writer access through a named mutex. Foreign keys and synchronous FULL are enabled. Default journal mode is DELETE; WAL is an explicit local-store option. Direct cross-thread writes are rejected. SQLite 3.42.0 with FTS5 was measured by the storage tests.

Schema 9 is created through nine checksum-verified migrations. Existing migration text is immutable. A newer incompatible schema is rejected safely. Failed upgrade DDL and ledger changes roll back, and an upgrade from an existing schema makes a recoverable pre-migration database backup. Interrupted running generations recover as failed, preserving the previous published head.

ObserveManifest records current fingerprints independently of the capture. BeginGeneration freezes configuration and manifest identity. Syntax and semantic facts are staged in transactions. Before publication, inputs and configuration are revalidated. Only a coherent complete generation becomes the current head. Partial, failed or superseded captures cannot clear the exact-input stale state or overwrite a prior current head.

Evidence and immutable history

Normalized tables include projects, configurations, source files, file revisions, generations, generation files, entities, entity versions, evidence, relationships, component properties, control-flow nodes and edges, knowledge records and revisions, links, review tasks, translations, artifacts, export jobs and audit events. Foreign keys tie evidence and relationships to their producing generation.

Triggers prevent mutation of terminal generations, source revisions, captured facts, configurations and authored knowledge revisions. Evidence ranges must fit the captured source bytes and use the generation analyzer version. Historical file and entity identities cannot be changed casually. A missing source is retained as missing; its historical annotations survive.

Modification detected, analysis stale, knowledge review needed, analysis failed and reassessment requested are separate persistent flags. A UI summary is a projection of these states, not a replacement for them. Parser success must not clear human review.

Background coordination and cached views

DPH.Workspace.Worker is a persistent TThread with bounded request and response queues. It initializes COM for passive MSXML parsing, owns database operations in the worker, and retains analyzer instances across requests. FMX polls responses with a timer, avoiding queued closures that outlive the form.

A stopped Delphi queue can report success while returning an empty item. Poll checks both the wait result and a non-nil response; shutdown tests guard against a destructor loop. Cancellation is checked between stages and marks a staged running generation failed when appropriate. It is not a promise of interruption inside every parser loop.

Workspace projections are managed JSON files in database.dphdb.views named by SHA256 and registered in artifacts. Cached opening verifies the managed path, size, checksum and generation. Cached facts are delivered before automatic verification when the original source entry is available; unavailable source remains detached. Connected checks compare manifest fingerprints and selection identity every ten seconds. Syntax and routine semantic caches are bounded and binding-context aware; conservative postprocessing remains separate from reused routine facts.

FMX navigation and visualization

DPH.UI.Main binds its published controls to the FMX resource. Explorer queries are paged at 250 rows; data filtering supports kind, text and qualified scope. History and bookmarks use canonical keys, with surviving same-project bookmarks retained across reassessment. Inspector output retains source metadata and serialized properties. Source memo navigation is permitted only after live hash validation.

OpenSelection queues a candidate without replacing the current database, source or workspace. PollTimer installs it only after the worker returns accepted captured evidence. The worker preflights source identity before creating a database; cached opens check captured XML ProjectGuid and Pascal program declarations before returning a view. Rejections retain the current workspace, and rejected startup caches show Getting started. Inventory identity also covers referenced project groups. DatabaseDialog.InitialDir is the design-time relative default; startup resolves it under the deployment root, creates that folder, and applies the same path to OpenCacheDialog. Distribution packaging copies only the Databases instructions, never its databases or views.

Graph paint boxes draw bounded evidence data with palette and geometry values supplied by design-time style rectangles. Screen reconstruction draws approximate source-derived rectangles without creating components. Its gallery is bounded to 60 forms. Runtime styles, custom painting and inherited resources are not silently fabricated.

Keep lifecycle and ownership explicit: the view owns its JSON document, UI arrays own their records, responses transfer document or knowledge ownership to the main form, and the worker frees unconsumed responses during shutdown. Guard unsaved authored text before changing projects or replacing editor state.

Curated knowledge and review

SaveKnowledge requires title, body, a named author, a supported kind and a captured generation. Existing records require their expected revision and stable kind. Ordinary titles stay stable; structured glossary names can change while their identity and history remain intact. Each edit appends a needs_review revision, preserves links and opens a pending task. Knowledge bodies are bounded to 1 MiB. Authored text is never regenerated from parser output.

ReviewKnowledge requires the expected current revision, a named reviewer and a matching captured source/configuration. The connected worker observes current source before accepting a review. A missing linked entity cannot be verified under a replacement name. Verification, rejection or retirement appends a revision and records attribution; history remains append only.

Manifest and configuration changes conservatively invalidate active knowledge, derived artifacts and translations. A partial static capture may support review of an attributable statement against available evidence, but its partial coverage remains visible. Local names are not authenticated enterprise identities. Human verification and automatic static derivation remain different evidence categories.

Canonical publication and privacy

DPH.Publish.Model constructs one versioned documentation model from a captured workspace and current curated knowledge. It carries generation, configuration, input/context fingerprints, detached/stale state, symbols, typed relationships, control flow, source references, human history, coverage and explicit unknowns. Full and task-scoped views share those facts.

Task scope selects the requested identities and complete direct incident context. It rejects missing or excluded seeds and rejects an oversized task rather than silently truncating a full export. External source roots and sensitive filename categories are excluded by default. Explicit relative prefix exclusions remove file-derived symbols and relationships. Source snapshots and generated AGENTS.md are absent by default.

Redaction recognizes common credential assignments, private keys, AWS access keys and JWT-shaped tokens throughout string data. It is defense in depth, not a guarantee that every project-specific secret format is recognizable. Require explicit exclusions and a disclosure review for sensitive content. No translation or AI provider is contacted by normal publishing.

HTML guides and bundle validation

DPH.Publish.Export writes a new destination outside source. The offline HTML uses encoded imported text, local search and styles, internal anchors, print CSS and a restrictive content security policy. The default WebView2 policy permits only the authorized local index and about:blank; publication explicitly enables hash-named object pages in its managed objects folder. Help retains the index-only policy; learning explicitly permits content-hashed pages only inside its owned preview directory. There is no arbitrary native bridge. Data does not supply executable markup.

JSONL indexes contain symbols, relationships, forms/events, data usage, configuration, flow nodes, knowledge and evidence references. Manifest and checksum files accompany canonical.json. The validator checks filenames, hashes, expected input fingerprint, canonical identity, source hash/range, generation and cross-reference integrity. A stale manifest is rejected by default. Unsigned hashes establish integrity relative to the index, not author authenticity.

Editable engineering and user DOCX guides derive from the same sections and evidence model. They include title, last changed capture identity and a changeable TOC field. LibreOffice provides companion PDFs; the project rendering tool verifies cached page numbers and renders page images for layout inspection. Missing business procedures stay explicitly absent rather than being invented.

Translation boundary

IDPHTranslationProvider exposes provider identity, version, external status and prose translation. Protected segments retain identifiers, SQL, paths, hashes and protocol keys byte for byte. Cache keys include source content, revision, source/target locales and provider/version. Changed prose produces a fresh pending entry; named review applies to the exact cached translation.

TDPHAtlasAdapter reads explicitly selected DAT JSON packs offline. The independent Apache-2.0 reader snapshot and license reside under dependencies\DelphiAppTranslation; VCL2FMX is unchanged and is not a build or runtime dependency. No network provider is configured. Tests exercise actual packs, persistent cache, terminology changes, source invalidation and exact-content named review. The offline GTranslate region is disabled, with local bilingual labels and sorting; do not activate network translation merely because a generic widget name appears in documentation.

Verification and release discipline

Run the phase build/test scripts elevated with the installed toolchain. WP0 checks FMX resource ownership; WP1 checks storage, crash recovery and migrations; WP2 checks inventory, encoding, syntax and source safety; WP3 checks semantics and pinned reference traces; WP4 checks workspace/background behavior; WP5 checks immutable authored history and pending review; WP6 checks publication, exclusions, sanitization, hashes and translation protection.

Use exact input hashes before and after analysis, including failures. Preserve original reference and instruction files. Keep tests that demonstrate invariants rather than tests that merely mirror implementation. New source, validation failure or unresolved risk justifies another targeted test; a passing unchanged phase does not need endless repetition.

H: development backups are enabled. Allow at least 60 seconds for this USB drive to wake before assessing availability. Git remains enabled: review intended changes, commit validated work and push both public and private remotes without force or history rewriting. Remove a stale zero-byte index lock only after verifying no Git process runs. WP7 records measured performance, portability, security and explicit acceptance evidence. Independent first-time programmer acceptance and unknown historical business intent cannot be invented by automation.

In the historical pre-rework benchmark on the recorded Core i7-1165G7 host with eight logical processors and about 16 GB RAM, the pinned reference pipeline measured 82.6 seconds for full capture, 2.87 seconds for cached opening and 75.9 seconds for incremental reassessment. The message-pump maximum gap was 64 ms. Peak test-process working set was about 1.21 GiB, including repeated captures, JSON cloning and FMX/WebView2 tests. These are baseline measurements, not agreed hardware-independent budgets. Large syntax-cache fallback and conservative semantic recomputation explain the modest reassessment speedup. Formal acceptance retains independent complex-flow review, missing business/user content, AC-25 and performance-budget agreement as outstanding.

Current semantic cache and extended evidence

The current analyzers are DPH-WP2-5 and DPH-WP3-11 with IR and semantic schema 1. Routine semantic cache reuse is bounded to 512 records and 32 MiB. Binding context hashes invalidate affected reuse. Synthetic startup and postprocessing occur after reusable routine facts, preserving their relationships to current identities. This is bounded evidence reuse; it is not a proof of an optimal incremental solver.

Supported extensions include typed aliases, inheritance and accessors, startup entry and unit phases, case/goto/with control flow, asynchronous operation handles and captures, cancellation and event waits, formal arguments and result producers, static SQL projection, output-writer lineage, lifetime and exception relationships, explicit test-call links, and qualified form navigation. Every relationship retains resolution, source range/hash and rule. Unsupported grammar, aliases and dynamic behavior remain diagnosed or qualified.

Current schema and authored document contracts

Schema 9 uses checksum-verified migrations 001 through 009. document_section_versions associates immutable authored revisions with stable section ID, document kind, audience and source generation. knowledge_revision_links preserves ordinal, target kind/identity, captured generation, role and label. knowledge_questions preserves the exact deterministic question identity and captured JSON associated with a response record.

SaveKnowledge appends immutable text and link snapshots. Empty LinksJSON preserves the prior snapshot; an explicit JSON array replaces it. Entity, relationship and evidence targets remain tied to generation identities; screenshot targets reference managed artifacts; section targets pin an exact knowledge revision. Current reverse section references open review when their targets change. Verify rejects obsolete rich-link generations or target revisions.

Question identity hashes kind, entity key, range, source hash, prompt, rule and context fingerprint. Generation is retained separately, allowing unchanged evidence to keep its question identity across captures. An answer needs actual text after Answer. Only verified content with no pending task suppresses an exact question. Export converts question metadata to typed JSON before recursive redaction.

Native review and learning projections

ReviewTasks provides bounded pending tasks, prior and current authored text, baseline/current configurations, affected object signatures, types, source hashes and relationship comparisons. A missing baseline is reported. Native controls invoke existing optimistic-revision and connected-source review checks; they do not create a separate approval authority.

RoutineLesson uses the routine CFG and an indexed outgoing-edge lookup. It explains source points, actual expressions and destination lines, with explicit loop, branch, early-return, cleanup and exceptional semantics. The same lesson is used by ObjectText and ObjectHTML; exported document prose and offline object pages therefore share its source interpretation. Its bounds are 60 points and eight outgoing paths per point.

Preview HTML files have immutable content-hashed names, tracked for cleanup. BrowserPolicy allows them only in the owned preview directory when explicitly enabled. This avoids rewriting a file held by WebView2. Arbitrary filenames and external navigation remain blocked. Native browser tests render the app pages; they do not execute the analyzed application.

Saved observation persistence

ImportObservation accepts explicitly selected UTF-8 text extracts up to 256 KiB with named contributor and run context. It checks generation and optional entity existence, captures original byte SHA256 and linked file-revision SHA256, and stores content plus attribution in a managed hash-named JSON artifact of kind runtime_observation. No schema migration is needed for this artifact kind.

ObservationsJSON verifies managed identity, size, checksum and generation before returning the newest 50 records. The response owns the array until PollTimer transfers it to the form. LoadView clears old observation rows; historical records cannot navigate as current. Saved text stays in a native read-only memo rather than an executable browser surface.

ManagedArtifactPaths includes observations for explicit portable backup. Retention protects their associated captured views. Publication currently omits observations rather than automatically disclose potentially sensitive logs. Contributor claims do not set runtime_observed on static analysis or create an automated causal trace.

Regression commands and evidence

Run commands from the project root in an elevated Command Prompt so RAD Studio can read its environment. tools\build-wp0.cmd builds the app and checks FMX bindings. tools\validate-completion-wp1.cmd serially builds app, catalog, lifecycle, workspace, knowledge, selection and startup tests, then removes recognized disposable fixture directories.

Run tools\build-wp1.cmd for 46 storage checks, tools\build-wp2.cmd no-report for discovery/resource checks, and tools\build-wp3.cmd to compile semantic tests. Then explicitly run bin\Win32\Debug\DPH.Tests.Semantics.exe "C:\New Delphi Projects\Delphi Project Handoff"; the build script alone does not run semantic assertions.

tools\build-learning.cmd exercises managed images, cached supplemental-evidence recovery, actual WebView2 rendering, source walkthroughs and collapsed CFGs. --no-ui isolates deterministic checks and is not native acceptance. tools\build-wp6.cmd checks canonical scope, redaction, link integrity, publication and translation protection. Use exact log files and current input hashes for claims; historical reports are not automatically refreshed acceptance.

A native-learning test once dereferenced an absent image array on a later background-verification response. It now checks the initial cached document/image response directly. Source attachment preserves supplemental-image warnings. Inspection disproved a proposed constructor race, and no worker thread-start change remains.

Rebuild and inspect these manuals

tools\build-guides.py generates the two application manuals from maintained operating content and current implementation hashes. Use the approved Python executable C:\Users\marti\AppData\Local\Microsoft\WindowsApps\python.exe. tools\render-guides.ps1 renders with LibreOffice, populates cached page numbers in editable TOC fields, renders again and writes companion PDFs under docs\pdf.

Inspect every final page PNG under bin\guide-qa for clipping, broken headings, unreadable code and TOC page numbers before delivering a guide. Rendered PNGs are QA outputs, not a mock application UI. Actual included interface images are identified by their capture source and fixture; they do not establish runtime behavior of a selected project.

Regenerate manuals after later translation, handoff-contract or source-navigation changes. The source evidence appendix records inspected file hashes, while docs\guide-source-validation.json provides a machine-readable manifest. A successful generator run alone is not proof that the writing, current behavior or layout is correct.

Business glossary implementation and checks

DPH.Knowledge.Glossary validates a business_glossary structured record with term, aliases, definition, examples and relevance/evidence context. Names normalize case and repeated whitespace. Terms/aliases are bounded to 128 characters, 32 aliases per term, definitions/examples to 16384 characters, context to 4096 characters and the record to 64 KiB. Unknown fields, duplicate fields and non-text aliases are rejected.

The existing migration 009 structured_json column stores the exact term fields on each immutable knowledge revision; no new migration is needed. SaveKnowledge enforces glossary kind, user document and all-reader audience, generated body agreement, project-local name uniqueness, type preservation and optimistic revision checks. Glossary titles can change while the knowledge ID and prior structured revisions remain intact. Retired records continue reserving their current names.

DPH.UI.Glossary.fmx defines every term editor control for the IDE/Object Inspector. DPH.UI.Main.fmx defines the glossary list, filter, detail memo and commands. Apply prepares a draft in the existing Knowledge pipeline. The modal editor retains a cloned saved revision so a background refresh cannot invalidate its input pointer. Current source links, explicit review, search and stale-source tracking use existing repository contracts.

DPH-LEARNING-8 publication emits a dedicated HTML glossary index and indexes/glossary.jsonl from the canonical knowledge records. Both user and engineering DOCX use those same records. Validate compares the glossary JSONL with the canonical glossary subset; recomputing the artifact hash cannot conceal divergent content. Older publication versions remain readable under their prior contract.

From an elevated Command Prompt run cd /d "C:\New Delphi Projects\Delphi Project Handoff", then tools\build-glossary.cmd. The suite uses the fixed ignored bin\glossary-validation directory and controlled fixtures. It checks field validation, normalized conflicts, persistence, rename, immutable revisions, stale edits/reviews, isolation, source-linked invalidation, search, publication agreement/exclusions/tampering and IDE-designed native form workflows at normal/minimum sizes. Native images are FMX PaintTo renders of actual instantiated controls; they are not screenshots of the analyzed target application.

Build the application with tools\build-wp0.cmd. Run tools\build-wp5.cmd for the existing knowledge/link/review regressions and tools\build-wp6.cmd for publishing/translation regressions. A FAIL or nonzero exit means the corresponding check failed. See docs\phase1-glossary-validation.json and the glossary completion note for actual current run evidence.

Offline translation and localized publication

Migration 008 adds the persistent translation cache key and JSON record. Cache identity covers saved knowledge revision, requested locales, provider pack hash, source segments and terminology. SQL state overlays cached metadata so source invalidation remains authoritative after database reopen. Approval requires current source review, complete translated prose and a named reviewer; it never updates authoritative source text.

TDPHAtlasAdapter uses DAT.Runtime.LanguagePack plus its two copied core dependencies, original license and hashed snapshot provenance. Selected packs are bounded to 4 MiB, strictly UTF-8 and application scoped. Exact source phrase lookup and base-language fallback are explicit. Common code, SQL, paths and identifiers are protected during segmentation; unusual unmarked identifiers still need human review.

DPH-LEARNING-7 publication writes localization.json and, when a locale is requested, a hashed HTML page and Localized_Sections.docx. Verified exact-revision cache entries are eligible only while the canonical source section is reviewed and has no pending tasks. Redacted source bodies use source fallback. The supplementary document explicitly limits its scope to authored bodies.

Source navigation is separately opt-in. Only scoped captured text with matching source hashes is rendered as encoded, redacted HTML. Task masks use captured noncontainer ranges and original line starts. source-navigation.json records included line numbers and omission reasons. Validate checks scope, file identity, line order/counts, generations and disclosure metadata. No original source snapshot is shipped.

Native validation and release evidence

The design-time Validate bundle controls dispatch validate_bundle to the worker before a database connection is required. The owner supplies an expected fingerprint independently. Stale rejection is default and historical inspection is an explicit qualification. Native reports distinguish integrity from human approval.

The Win32 project enables DCC_LargeAddressAware in its IDE project options. The hardening and learning harnesses use the equivalent PE flag. The expanded pinned corpus exposed an out-of-memory failure after repeated captures and native workspace loading under the former address limit. Keep memory measurements and actual large-corpus validation with this change; the linker flag does not make memory consumption unbounded or establish a release budget.

Run tools\validate-wp7.ps1 elevated for the pinned reference hardening suite and before/after hash proof. It reads the current schema constant rather than an obsolete hardcoded version. The suite measures full capture, cached open, reassessment, message-pump gaps and peak process working set on recorded hardware. Measurements require owner-agreed budgets before release acceptance.

The independent first-time programmer and source-review worksheet is docs\WP7_Maintenance_Exercise.txt. Preserve actual participant answers, source citations, uncertainty judgments, assistance, timings and defects. Do not substitute automated tests or prepared instructions for AC-25 or independent explanation review.

Structured project profile contract

TDPHWorkspaceView.ProjectMembers projects captured syntax files into a selected-scope catalog without filesystem discovery. DPH.UI.Main.fmx defines separate Known databases and Selected project subpages, with a native list and evidence memo. Program, library and package declarations retain source references; project properties and references retain captured values and child conditions. Parent conditions may apply even when a child has none recorded. tests\DPH.Tests.ProjectCatalog.dpr checks a selected group, missing referenced project, output kinds, settings, native selection and input immutability.

Migration 009 adds an object-valued structured_json field to append-only knowledge revisions. TDPHProjectProfile validates 15 known text fields, required purpose and attribution, per-field bounds and a 64 KiB record bound. SaveKnowledge enforces stable project scope, title, kind and generated body, prevents ordinary-record conversion, duplicate profiles and loss of typed data, and preserves structured fields during named review and continuity revisions. Publication decodes typed metadata before recursive redaction, including revision history.

DPH.UI.ProjectProfile.fmx defines every editor control at design time. Apply returns an attributed draft to the existing Knowledge save pipeline. It does not bypass unsaved-edit guards or automatically approve

information. tests\DPH.Tests.Phase1.dpr verifies persistence, review, stale editors, history, field bounds and native form behavior; publishing regressions check canonical, HTML and DOCX inclusion.

Quality analysis and revision comparison

DPH.Analysis.Quality emits DPH-QUALITY-1 evidence during workspace capture. Routine metrics, eligible token-body equality, conservative missing-inbound and disconnected-CFG findings, and iterative call-graph strongly connected components use captured inputs only. The worker invalidates an older connected cache without this analyzer version. All findings remain qualified candidates.

DPH.Workspace.Comparison compares two hash-verified retained captures from the same project database. It rejects differing selection and syntax or semantic analyzer contracts. Deterministic identity indexes distinguish file changes, symbol additions or removals, captured structure changes and file-only changes. Routine-local flow numbering avoids global node-number shifts. Results use filtered 200-row pages; individual structure displays are limited to 4000 characters with full-content structure hashes and explicit truncation.

Comparison runs on the existing background worker. It reads retained database artifacts rather than current selected source files. Native comparison controls and evidence panes are declared in DPH.UI.Main.fmx. tests\DPH.Tests.Comparison.dpr checks literal-expression changes, unchanged neighboring routines, flow-edge rewiring, identity changes, contract rejection, pages, native panes, worker loading and missing caches.

The Phase 1 traceability ledger preserves the supplied requirements and limitations. Source comparison remains partial until optional historical source retention supports actual source-text differences; captured structural comparison must never be described as reconstructing those bytes. Human business answers and independent first-time-user acceptance remain separate evidence.

Source and revision evidence

The following source files and resources define this guide. SHA256 values identify the inspected implementation. Rebuild the guides after validated changes and retain the associated acceptance evidence.

DPH.Analysis.Async.pas

`876a836f25b3d323ac4d6f635701fb4e8e97955fdff39c964368aa62f1a4c153`

DPH.Analysis.Explanations.pas

`be976f2199be2c8e86a0cf8be936d008aa83739cf116c3e81ca68bf3692fc8e7`

DPH.Analysis.Flow.pas

`ee7610544411a64da399659f2af11e84408d285837a45205d610d2c37af8ef57`

DPH.Analysis.Model.pas

`67383aae374aabba6832526e9d373827a5133e4369a61eaad9cffef9f239ecdb`

DPH.Analysis.Persistence.pas

`08609c3df54250a8f6712e22e991f7ad8d444352e7103ba0959d2feba4a2b3c7`

DPH.Analysis.Quality.pas

`50196d33a8d15cfeb52528dde93a2bfdc22b153e2533830b08e3ac097e14fd82`

DPH.Analysis.SQL.pas

`68ea587d47e2c7ebf660f85eed6b605f6076480ead3a461742b8290fec7a130e`

DPH.Analysis.Symbols.pas

a362f3a2b46bdc77883c129efd1c14192d6128ecb616f82915222df370d2d006

DPH.Knowledge.Glossary.pas

9c50a06ccf405f786526e206e5e84ea956604eeb42e1a84b9f91c31cced70dcb

DPH.Knowledge.ProjectProfile.pas

e901a58b02c2aafe2cc21df4da7dced2807a9cc8840c325c72d9db64cedc44bd

DPH.Parse.Forms.pas

8b0a82186f7831bd1c27bf377409ac6e49b77c228666e6087fca8758c85d4cc7

DPH.Parse.Lexer.pas

6ce5bbc66f8170b17eaeda536b8e500ca9992131477c09bc8e1a33bb6dd1c7f0

DPH.Parse.Model.pas

1bc634e14dbc4d2b2b94d566fcca9eae7911fd1299b70ef7378dcba577c80778

DPH.Parse.Pascal.pas

016adbe9ba24b90d457455722782cdbbc657233bd1a07ba1bcdcd77f3340587e

DPH.Parse.Preprocessor.pas

34b7513019a41bc9911babfda72041addd563959ee54c54d0759b8cc9fa7fefc

DPH.Parse.Resources.pas

f63153edba8656b315956fdd8ccf52e45b1446d8192f9b84c96c153bf676258f

DPH.Project.Analysis.pas

5af3808df19a7c403b456342e45b9e2c8c63195368847b151f3c57ed461b5c2a

DPH.Project.Inventory.pas

061736891f0fab486996ff5d8132af1cf5e25a2f93a74fb3e03ad8fa04e2c266

DPH.Project.Source.pas

8efa444743f0e90831631df963f26a2542561efab63b26f1c5b41fff6b96994e

DPH.Publish.BrowserPolicy.pas

74d2f36c879f1b7b55133241c3e1abc28ce113c4d9f581a5123be83f94d705ce

DPH.Publish.Export.pas

0a36aa00c6c9591521d53977e2a4985b4cb390dfa79c7dcbc337c74ff306c39e

DPH.Publish.Model.pas

924538cea0f766a19094f0fd41ff4381094e905f1a2e7db9474e3b6437bad292

DPH.Publish.Translation.pas

978be4250a2332736f73cd4e56e85d5c2f0b0d3a6224a4db2f8f3b7b5f4cde20

DPH.Storage.Backup.pas

78e8c8de8af5120f8ce453793f947c5e2918e51dafc66f9e129c964fe28697bc

DPH.Storage.Database.pas

28e17f8ddf7196406c1ef3ba3ce4e98959764660d90cec0ae7abd249d0043b33

DPH.Storage.Types.pas

1e83ccbe6868979ff9d763a4f2bf4d64691c2b2343de6717391f03e204d5d219

DPH.UI.Glossary.pas

```
fb15295b29a9bbb9caa05cea723e737c81b2418a271f59fb6071dbffe37f8741
```

DPH.UI.Main.pas

```
f4495d86b4a6390e3cf24f645d73ad8f69e5fe03af79aab9e5fdadcb78caa345
```

DPH.UI.ProjectProfile.pas

```
f3fe7ed94987e956999e7449887cd8ca259f009b3d4ac934c1e8e09b8e9fab59
```

DPH.Workspace.Catalog.pas

```
5fed34d1919dfd5b34216be36f92fc1764847a65d62112613b2a0e1d44a05940
```

DPH.Workspace.Comparison.pas

```
d4953d401b70fbe65b8b9b9fe387431cb7e2c84afc5dab5c9170deafe9ceacce
```

DPH.Workspace.Graphs.pas

```
6fd8b84644df65a0746aaad3598dad9406fa2d55e637e1e30f2175cefd1be911
```

DPH.Workspace.Model.pas

```
03bbe5a9dc0943758be56898659bff9381ccb0707cf22a449e5c8e158a8b82bf
```

DPH.Workspace.Screen.pas

```
68305e6ec36664e76d05b8c9af21416dd51a32116d47ea55156977b4c10855de
```

DPH.Workspace.Worker.pas

```
3818fff9f49e530546653f7ad39108a9ee83f79de69ead6306894ef8ddc1e2ab
```

DPH.UI.Main.fmx

```
6212567a7d5b8c1cd18a4f5b2ca0bb99c0deac95db5723ec633f1413f5b5206b
```

DPH.UI.ProjectProfile.fmx

```
62e7fef97972fdb29f1a878e70a99d8284b6f7d8cfbddb9f625bb535728c2142
```

DPH.UI.Glossary.fmx

```
3334c41ac643daa3596f1a5c9cb8fb65c3447daf77cf00006d6a0ca7a9d517d5
```

DelphiProjectHandoff.dpr

```
2b922d5561c2ab3e2b65590583f646e29fc12c8b5c36a08fe401a6ebed90d3f9
```

DelphiProjectHandoff.dproj

```
717637f3a1d07f33e0902beb1da0d2765ba420ccbe5383db9c5bab082ee16977
```